

Bon, je vais répondre thématiquement, en commençant par ce qui est objectivement prioritaire selon les profils.

Ce que dit vraiment le profil

Le gros problème n'est pas dans les kernels CUDA — c'est le pipeline de données. nsys le dit clairement : **70% du temps CUDA API est du cudaMemcpy HtoD**, soit 7807 appels à ~207µs chacun. Tes sgemm cuBLAS sont déjà très bien (76% throughput, "compute and memory well-balanced"). Le vrai levier n'est pas là où ncu te le suggère.

Les suggestions ncu sur `kernel_update_weights` (grid size 1-2, 0.01 waves) sont réelles mais ce kernel ne représente que 3.3% du temps GPU total — c'est du polish, pas du gain structurel.

Priorité 1 : dataset entier sur GPU

50k × 784 × float32 = ~157MB. Labels : ~200KB. Ton RTX 4050 a 6GB. Tu charges les deux une seule fois au début avec `cudaMalloc` + `cudaMemcpy`, et ensuite tu ne fais que des opérations device-side dans la boucle d'entraînement (slicing via pointeur arithmétique). Ça élimine la source dominante de stall.

C'est aussi "vendable" dans le sens où ça illustre exactement la décision trade-off mémoire/latence qu'on fait en prod. Sur ton cluster 4090 avec 24GB par GPU, même raisonnement s'applique à un dataset beaucoup plus grand.

Priorité 2 : kernel fusion bias+relu

Actuellement tu fais deux passes DRAM distinctes pour `kernel_bias_forward` et `kernel_relu` — chacune memory-bound à 64-75% DRAM throughput selon ncu. C'est le cas parfait pour la fusion : une seule passe lit les activations, ajoute le biais, applique ReLU, écrit. Le gain théorique est ~2x sur ces kernels (qui représentent collectivement environ 15-20% du temps kernel).

C'est l'exemple textbook de **kernel fusion** de ta liste de cours, et c'est exactement ce que cuDNN/JAX/PyTorch font en interne. C'est clairement démontrable et mesurable avec ncu avant/après.

Priorité 3 : `kernel_update_weights` thread coarsening

Pour les biais (`OUTPUT_SIZE=10`, `HIDDEN_SIZE=512`), tu lances 1-2 blocs sur 20 SMs. Fix simple : un seul kernel `update_all_weights` qui loop sur tous les paramètres, ou au moins fuse les 6 appels en un. L'impact absolu est faible mais ça montre que tu comprends l'overhead de kernel launch.

Accuracy silencieuse — à vérifier

Ta suspicion est légitime. Le point de friction : cuBLAS est column-major. Ton `output` après `cublasSgemm` est stocké comme `OUTPUT_SIZE × BATCH_SIZE` column-major, ce qui est équivalent à `BATCH_SIZE × OUTPUT_SIZE` row-major. `softmax_kernel` accède `x[b * size + i]` avec `size=OUTPUT_SIZE` — ça correspond. Mais `kernel_compute_output_gradients` fait pareil sur `grad_output`. Vérifie que ce tenseur est cohérent avec ce que `cublasSgemm` attend ensuite dans le backward.

La façon simple de valider : compare la loss epoch 0 vs une implémentation PyTorch équivalent. Si epoch 0 diverge de >10%, il y a un bug de layout.

Ce qui rend le projet vendable

Le problème avec "MLP sur MNIST en CUDA" c'est que c'est connu. Ce qui change la perception :

Narrative progression visible sur le repo : Python baseline → C → CUDA naïf → CUDA optimisé (tiled GEMM, cuBLAS) → kernel fusion → transformer FFN → multi-GPU. Tu as déjà plusieurs étapes. La prochaine marche naturelle est de **transformer le MLP en un FFN block de transformer** : même architecture (Linear → activation → Linear), mais tu remplaces ReLU par GELU (un kernel de plus), tu ajoutes une couche LayerNorm fusionnée, et soudainement tu as les primitives d'un GPT-2 FFN. C'est exactement ce que fait Andrej Karpathy dans `llm.c`, et c'est très référencé dans les entretiens ML systems.

Ce que ça montre concrètement à un recruteur Cohere ou similaire : tu sais pas juste "faire du CUDA", tu comprends comment les blocs de LLM training sont assemblés depuis les primitives.

Multi-GPU sur ton cluster 4090

En passant à 4× RTX 4090, le changement principal c'est la **data parallelism** : chaque GPU traite un shard du batch, et tu dois allreduce les gradients après le backward. Sans NVLink, tu es sur PCIe Gen4 x8 qui te donne ~16GB/s bidirectionnel par GPU. Pour un modèle de cette taille, c'est négligeable — le bottleneck reste le compute.

Ce qui est intéressant à implémenter manuellement : un **ring-allreduce** sur NCCL (ou même à la main avec CUDA IPC), qui est exactement le mécanisme sous-jacent de ZeRO-1/DDP de DeepSpeed. Si tu veux un angle "je comprends DeepSpeed de l'intérieur", c'est par là que ça commence — pas ZeRO-3 complet, mais juste data parallel avec gradient sync correcte et mesure du overhead communication vs compute.

Quantification

FP16/BF16 mixte est beaucoup plus intéressant que INT8 pour ce contexte. Tu remplaces `float` par `__half`, utilises `cublasHgemm` ou `cublasSgemmEx`, et tu dois gérer le **loss scaling** (les gradients underflowent en FP16). Ça te donne directement le vocabulaire de l'entraînement LLM mixte précision. Sur RTX 4090 (TF32/FP16 tensor cores), tu verrais un vrai gain de throughput mesurable.

INT8 quantization pour l'inférence c'est un angle différent — plus proche de ce que fait vLLM/AWQ que de l'entraînement.

Connexion à l'inférence LLM

Le lien le plus naturel et le plus crédible : ton forward pass CUDA est un inference pass. Si tu l'étends à un transformer (attention + FFN), tu as un moteur d'inférence minimal. La différence avec vLLM c'est la gestion de la KV cache et le continuous batching — tu peux décrire exactement pourquoi ces problèmes n'existent pas dans un MLP et pourquoi ils apparaissent avec l'attention causale. Ça montre une compréhension systémique.

Ce que je ferais dans l'ordre

1. Dataset sur GPU → mesure du speedup
2. Fusion bias+relu → benchmark avant/après ncu
3. Vérifier la correctness accuracy vs PyTorch
4. Remplacer la couche output par une architecture compatible avec un transformer FFN (GELU, deux couches)
5. Ajouter LayerNorm fused kernel
6. Benchmarker sur un 4090 du cluster avec le même code

Chacune de ces étapes est un commit propre sur le repo avec les chiffres dans le README.
C'est ça qui rend le projet lisible en 2 minutes par un reviewer.